arXiv:2512.18088v3 [cs.SE] 5 Jun 2026

Detecting Flakiness in Quantum Software: A Dynamic Testing Approach

Dongchan Kim[a,1], Hamidreza Khoramrokh[b,1], Lei Zhang[a], Andriy Miranskyy[b]

[a] *University of Maryland, Baltimore County, Baltimore, MD, USA*
[b] *Toronto Metropolitan University, Toronto, ON, Canada*

Abstract

Flaky tests, tests that pass or fail nondeterministically without changes to code or environment, pose a serious threat to software reliability. While classical software engineering has developed a rich body of techniques to study flakiness, corresponding evidence for quantum software remains limited. Prior work relies mainly on static analysis or small sets of manually reported incidents, leaving open questions about their prevalence, characteristics, and detectability.

This paper presents the first large-scale dynamic characterization of flaky tests in quantum software, focusing on the Qiskit TERRA core library. We executed the Qiskit TERRA test suite 10,000 times across 23 releases in controlled environments. For each release, we measured test-outcome variability, identified flaky tests, estimated empirical failure probabilities, analyzed recurrence across versions, used Wilson confidence intervals to quantify rerun budgets for reliable detection, and mapped flaky tests to TERRA subcomponents.

Across 27,026 fully qualified test identifiers, we identified 62 unique flaky tests. Although overall flakiness rates were low (0–0.17%), recurrence was substantial: 52 of 62 flaky tests (83.87%) reappeared in multiple releases, while only 10 tests (16.13%) were confined to a single release. Empirical failure probabilities spanned several orders of magnitude, with a median of 9×10^{-4} and 34 tests (54.84%) at or below 10^{-3}, implying that thousands to tens of thousands of executions may be required for confident detection.

These results show that quantum test flakiness is rare but difficult to

[1] These authors contributed equally to this work.

detect under typical continuous integration budgets. To support future research, we release a public dataset of per-test execution outcomes.

Keywords:
quantum flaky tests, dynamic testing, quantum software testing, quantum software engineering

---

1. Introduction

Flaky tests, i.e., tests that unpredictably alternate between passing and failing despite no changes to code or environment, pose a persistent threat to software reliability (Luo et al., 2014; Micco, 2017; Memon et al., 2017). Their presence reduces trust in test outcomes, obscures real defects, and slows continuous integration (CI) or continuous deployment (CD) pipelines. In classical software, extensive research has shown that flakiness arises from diverse sources such as concurrency, nondeterministic execution order, platform dependencies, and environment-related timing issues (Gruber and Fraser, 2022; Parry et al., 2021). To mitigate these challenges, the classical literature has developed a spectrum of techniques: dynamic re-execution strategies (Alshammari et al., 2021; Lam et al., 2019b; Bell et al., 2018), static analysis (Luo et al., 2014; Gruber et al., 2021; Lam et al., 2020, 2019a; Parry et al., 2021), and machine-learning-based predictors (Ziftci and Cavalcanti, 2020; Bell et al., 2018; Lam et al., 2019b; Dutta et al., 2020; Alshammari et al., 2021; Verdecchia et al., 2021).

As the quantum computing ecosystem matures, similar reliability challenges have begun to surface (Murillo et al., 2025). Frameworks such as Qiskit (Javadi-Abhari et al., 2024) now provide full software stacks and have testing infrastructures comparable in scale to classical systems. Yet, quantum software introduces additional sources of nondeterminism: probabilistic program semantics, noise-aware compilation flows, floating-point approximations, evolving toolchains, and constraint–sensitive optimizations in the transpiler. Early studies have identified flaky tests in quantum software repositories (Zhang et al., 2023), but current work relies primarily on static analysis of small, manually curated, incident-report-driven datasets (Zhang and Miranskyy, 2024; Kaur et al., 2025). Thus, we still lack a systematic, empirical understanding of how frequently flaky tests occur in quantum software, how they evolve across releases, and how difficult they are to detect given realistic rerun budgets.

Static machine-learning approaches depend critically on the availability of reliable ground truth. Without large, empirically derived datasets, especially those capturing very low-probability events, such models risk overfitting to limited historical cases (Verdecchia et al., 2021). Moreover, given that failure probabilities in quantum software can be extremely small, the limited rerun budgets typically used in CI pipelines may be insufficient to reveal rare nondeterministic behavior.

To close this gap, we conduct the *first large-scale dynamic study of flaky tests in quantum software*. We repeatedly execute the complete test suite for the Qiskit TERRA component[2] 10,000 times across 23 releases in controlled, containerized environments. This design surfaces rare nondeterministic failures that would remain undetected under conventional testing strategies and enables statistical analysis of empirical failure probabilities, temporal persistence, and subcomponent-level patterns. We also validate our initial findings and experimental settings with Qiskit developers (Qiskit Development Community, 2026a).

We formally state the *research questions* (RQs) guiding this study.

RQ1: How do the count and fraction of flaky tests change between TERRA releases?

RQ2: How persistent are individual flaky test cases across TERRA releases?

RQ3: What is the probability of detecting a flaky test given a specific number of executions?

RQ4: How are failure intensities distributed across releases?

RQ5: How are flaky tests distributed across TERRA subcomponents?

RQ6: How do flaky tests emerge, dissipate, or recur across the sequence of TERRA releases?

This work makes the following *contributions*.

[2] From hereon we will refer to it as TERRA. Its role is summarized in the project documentation: "This library is the core component of Qiskit, which contains the building blocks for creating and working with quantum circuits, quantum operators, and primitive functions (Sampler and Estimator). It also contains a transpiler that supports optimizing quantum circuits, and a quantum information toolbox for creating advanced operators." (Qiskit Development Community, 2025j)

1. *Large-scale empirical analysis.* We perform the first dynamic investigation that reruns the TERRA test suite 10,000 times across 23 TERRA releases (approximately 70 CPU-years of computation). This reveals how the prevalence and characteristics of flaky tests evolve over time.

2. *Probabilistic characterization of detectability.* We quantify empirical failure probabilities for flaky tests and derive confidence-based estimates of the number of repetitions required to detect failures of varying rarity. Our analysis shows that many quantum flaky tests exhibit extremely sparse failures, requiring a large number of reruns to surface reliably.

3. *Public dataset of flaky tests.* We release a curated dataset containing 62 unique quantum flaky tests, annotated with results from 10,000 controlled executions per release. This dataset provides high-quality ground truth for future research in quantum test reliability, debugging, and machine-learning prediction. It is available at `https://zenodo.org/records/20500432`.

By integrating large-scale dynamic evidence with probabilistic analysis, this study offers the first comprehensive view of flaky-test behavior in quantum software. The findings highlight the practical challenges in detecting low-probability nondeterminism and establish a foundation for more reliable quantum testing pipelines.

The remainder of the paper is organized as follows. Section 2 reviews related work. Section 3 describes the software under study, data-collection process, and execution environment. Section 4 presents the empirical analysis and answers the RQs. Section 5 synthesizes the findings and discusses their practical implications. Section 6 discusses threats to validity. Section 7 concludes and outlines future work.

2. Related Work

Flaky tests have been widely studied in classical software, and only recently examined in quantum software. This section reviews both bodies of work, highlighting the methodological gap that motivates our dynamic study.

2.1. Flaky Tests in Classical Software

Research on classical flaky tests spans three main areas: dynamic detection, static prediction, and root-cause analysis as discussed below.

Early work on classical flaky tests focused on detecting them by rerunning tests under varying conditions or orders. Techniques like iDFlakies (Lam et al., 2019b) repeatedly reorder and execute test suites to surface order-dependent flaky behavior, resulting in a dataset of 422 flaky tests. DeFlaker (Bell et al., 2018) detects flaky failures in real-time and flags failures not associated with recent code changes. Such methods remain effective but are computationally expensive, especially for large test suites.

To reduce rerun cost, several tools predict flakiness from code characteristics. For instance, Pinto et al. (2020) present a purely static approach that warns developers about flaky tests as they write code. FLAST (Verdecchia et al., 2021) represents test code in a high-dimensional space using bag-of-words features and applies sparse random projection with k-nearest neighbors. FlakeFlagger (Alshammari et al., 2021) combines static and dynamic features, supported by a dataset created by running 811 tests 10,000 times — far larger than previous dynamic efforts. They show that small rerun budgets (10, 100, and 1000 re-runs) detect only a minority of flaky tests ($\approx$ 26%, 45%, and 67%, respectively), motivating higher-budget studies.

Luo et al. (2014) provide a widely used taxonomy of ten flaky-test root causes, with asynchronous waits, concurrency, and order dependencies accounting for most cases. Eck et al. (2019) discovered four new causes of flakiness, including those related to test environments and unmet assumptions that had not been reported previously.

2.2. Flaky Tests in Quantum Software

Flaky tests in quantum software have received limited attention, with existing research relying almost exclusively on static or manual methods.

Zhang et al. (2023); Zhang and Miranskyy (2024) performed the first empirical study of flaky tests in quantum repositories by mining issue trackers and pull requests across 14 open-source projects. They identified 46 instances of flaky tests in 12 projects (0.3–1.8% of reported bugs), categorized eight common causes of quantum test flakiness, and documented seven common fix strategies. This work demonstrates that flakiness already exists in quantum ecosystems, but the scale is restricted by the availability of human-reported incidents.

Kaur et al. (2025) proposed the first machine-learning-based flaky-test detector for quantum software. Using a bag-of-words representation similar to classical static predictors (Verdecchia et al., 2021), they trained various classifiers that achieved strong performance on the limited set of available labeled cases. However, like classical static approaches, the accuracy of such models is fundamentally constrained by the lack of large, dynamic ground-truth datasets.

Sivaloganathan et al. (2024, 2026) proposed an automated framework for detecting flaky tests in quantum software by mining issue reports and pull requests using embedding-based similarity and large language models, expanding prior manually curated datasets. While their approach achieves strong performance for flakiness detection from textual artifacts, it remains dependent on developer-reported incidents rather than direct observation of execution-level nondeterminism.

2.3. Comparison with Prior Work

Table 1 positions our study relative to representative flaky-test studies in both classical and quantum software. The comparison is not intended as a direct numerical ranking, because prior studies differ in their units of analysis and data sources: static machine-learning studies depend on pre-existing labeled cases, whereas dynamic studies identify flakiness through repeated test execution. Instead, the table highlights the methodological gap addressed by this work. Prior quantum studies have shown that flaky tests exist in quantum software, but they do not provide execution-level ground truth, empirical failure probabilities, temporal recurrence patterns, or rerun-budget analysis. Our study complements these efforts by providing a high-budget dynamic characterization of flaky tests across 23 Qiskit TERRA releases.

2.4. Summary and Gap

Across both domains, dynamic rerun studies have proven essential for uncovering rare nondeterministic failures and for building reliable ground-truth datasets. Yet, no such large-scale dynamic analysis exists for quantum software. Prior quantum studies rely on incident reports or static features and therefore cannot: (i) measure true flakiness prevalence across versions, (ii) quantify empirical failure probabilities, or (iii) assess detectability under realistic rerun budgets.

Table 1: Comparison of representative flaky-test studies. The table contrasts prior work by domain, evidence source, detection method, scale, and rerun budget. N/A indicates that the study did not use a dynamic test-reexecution protocol.

Study	Domain	Evidence Source	Method	Scale	Rerun Budget
Luo et al. (2014)	Classical	Commit logs, bug reports, and patches	Manual/static root-cause analysis	201 flaky-test-fixing commits from 51 projects	N/A
Lam et al. (2019b) (iDflakies)	Classical	Test executions under different orders	Dynamic detection via test reordering	422 flaky tests from 82 projects	Variable; default 20 rounds; up to 56h/project
Verdecchia et al. (2021) (FLAST)	Classical	Test source code and existing labeled flaky/non-flaky test datasets	Static prediction using vector-space modeling, dimensionality reduction, and k-nearest neighbors	1,383 flaky tests and 26,702 non-flaky tests from 13 projects	N/A
Alshammari et al. (2021) (FlakeFlagger)	Classical	Repeated test-suite executions	Dynamic dataset construction + ML-based prediction	811 flaky tests from 24 projects	10,000 reruns/project
Zhang et al. (2023); Zhang and Miranskyy (2024)	Quantum	Issue reports and pull requests	Issue/PR mining and manual analysis	46 flaky-test reports from 12 of 14 projects	N/A
Kaur et al. (2025)	Quantum	Existing labeled quantum flaky-test cases	Static ML prediction	45 flaky Python files and 243 non-flaky Python files	N/A
Sivaloganathan et al. (2024, 2026)	Quantum	Issue reports and pull requests	Embedding-based mining + LLM classification	71 flaky-test reports from 12 of 14 projects	N/A
This work	**Quantum**	**Repeated test-suite executions**	**Dynamic characterization**	**1 project / 23 releases; 348 occurrences / 62 unique tests**	**10,000 runs/release**

This work addresses that gap by conducting the first large-scale dynamic flakiness study in a quantum software framework. Our dataset provides the empirical foundation needed for future statistical analysis and ML-based prediction.

3. Study Design

This study aims to measure and characterize flaky behavior in quantum software through large-scale dynamic re-execution. Our study design was adopted from the high-budget dynamic rerun protocol used in FlakeFlagger (Alshammari et al., 2021), which demonstrated the value of repeated execution for constructing reliable flaky-test ground truth. We adapt this experimental style to a quantum software setting by applying it across 23 Qiskit TERRA releases, using controlled containerized environments, and analyzing release-level failure probabilities, recurrence, and subcomponent-level

patterns.

This section describes the software under study, our data-collection pipeline, and the execution environment used to generate the dataset.

3.1. Software Under Study

We focus on the Qiskit software stack (Javadi-Abhari et al., 2024), one of the most widely used frameworks for quantum circuit construction, compilation, and analysis (Khan et al., 2025). Qiskit provides high-level circuit APIs, a multi-stage transpiler, simulators, backend interfaces, and a comprehensive built-in test suite, making it a representative and mature target for studying nondeterministic behavior in quantum software.

Our analysis concentrates on the core Qiskit library (Javadi-Abhari et al., 2024), formerly known as `qiskit-terra` prior to v.1.0, renamed `qiskit` thereafter. Its role is summarized in Footnote 2.

We study all 23 releases between v.0.25.0 and v.1.2.4 (see Table 3). The lower bound reflects an engineering constraint (the earliest versions rely on outdated toolchains that no longer build reproducibly) while the upper bound coincides with the last release supporting Python 3.8, which ensures compatibility across all versions under study.

3.2. Data Collection Method and Execution Testbed

Our goal is to obtain high-fidelity, release-specific measurements of nondeterministic test outcomes. To achieve this, we designed a pipeline that repeatedly executes TERRA test suite 10,000 times per release in clean, isolated environments.

Our execution protocol is designed to measure flakiness under a controlled, repeatable rerun setting rather than to maximize flake discovery through perturbation. We do not reorder tests, inject timing delays, or otherwise perturb the execution environment. Consequently, the observed flakiness rates should be interpreted as flakiness detected under this controlled protocol, not as an exhaustive estimate of all possible flakiness in TERRA. In particular, order-dependent or timing-sensitive flaky tests may be under-sampled by this design.

Qiskit uses TOX software (Tox Development Community) to virtualize the test environment and execute test cases on specific platforms. Since Linux is both Qiskit's primary development platform and the operating system of the high-performance computing (HPC) cluster that we used to run test cases, we selected it for our study.

Each run of the test suite was executed in a clean and isolated environment to ensure that the environment's state did not affect the tests. For each version, we prepared a dedicated Dockerfile that pins Python 3.8 and installs all dependencies needed to build and run that version's test suite. Because Qiskit depends on the Rust toolchain (Matsakis and Klock, 2014), we selected a compatible Rust toolchain for each release based on Qiskit's evolving minimum requirements. Although required libraries were cached in the Docker images, internet access was still necessary because tox, Qiskit's test orchestrator, resolves and verifies dependencies at runtime before executing the test suites.

Our HPC environment supports Singularity (Kurtzer et al., 2017) rather than Docker containers. Accordingly, each Docker image was converted to a Singularity image using the Singularity CLI. We then scheduled 11,000 executions per image, each using 4 CPU cores (Intel Broadwell processors) and 24 GB of RAM. Gathering the data on our testbed required approximately 70 CPU-years.

The slight over-provisioning (11,000 runs) provides resilience against transient issues such as network failures or dependency-resolution timeouts. After execution, we retained only the first 10,000 successful runs per release to maintain strict comparability across versions. This retained budget balances detection power and computational cost. It is consistent with prior high-budget dynamic flaky-test studies, such as Alshammari et al. (2021), while substantially exceeding the rerun counts typically feasible in per-commit CI.[3]

After the initial high-volume full-suite executions, we also performed targeted reruns as an internal validation step. We aggregated a master list of candidate flaky tests, including tests that required rechecking because of

[3]Smaller budgets would reduce cost but would make rare flaky failures much less likely to appear in the retained data. Larger budgets would improve detection but with rapidly increasing computational cost. We therefore selected 10,000 retained executions per release as a practical compromise: high enough to expose failures at the 10^{-4} empirical level when they occur in the sample, yet still tractable for a 23-release study that already required about 70 CPU-years.

We quantify this trade-off in Section 4.3.2. There, we derive the detection probability and use Wilson lower confidence bounds to estimate conservative rerun budgets. The analysis shows that 5,000 executions would often miss failures around the 10^{-4} level, 20,000 executions would still not provide confident detection, and confidence-based budgets can reach the order of 10^5 executions. Thus, 10,000 executions should be read as a high but tractable dynamic sampling budget, not as a guarantee of detecting all rare flaky tests.

possible infrastructure- or parallelization-related behavior. For each release, we then selectively reran only the candidate tests that were present in that version. This procedure was part of our experimental workflow rather than a proposed CI framework, but it illustrates a practical strategy for reducing unnecessary computation: broad discovery runs can identify candidate flaky tests, while subsequent validation runs can prioritize those candidates instead of rerunning the entire test suite at the same depth.

Each test suite produces text-formatted output in the standard `pytest` style. We collected all outputs into version-specific folders and parsed them using custom Python scripts. These scripts extract per-test pass/fail/skip counts and consolidate them into a structured CSV file. This process yields one CSV table per release, each summarizing approximately 20,000 test cases.

3.3. Dataset Description

All collected artifacts are publicly available at `https://zenodo.org/records/20500432`. Each of the 23 CSV files (one per release) shares a common schema summarized in Table 2. The fields describe per-test execution outcomes and the environment used for that release.

A test case is identified by its fully qualified `test_case_name`, consisting of the module path, class name, and method name. The 27,026 distinct test cases reported in Table 3 correspond to the union of these fully qualified identifiers across all 23 releases, not to per-version test instances. Thus, a test appearing in multiple releases is counted once in the "Unique" row if it has the same fully qualified identifier. For cross-release recurrence analysis, we treat tests sharing the same fully qualified name as the same test. This is an identifier-level definition of uniqueness, not necessarily byte-level or semantic equivalence of the test body across releases. The underlying test body may evolve through code edits, so a "persistent" flaky test could in principle correspond to a sequence of slightly different implementations.

We annotated flakiness per TERRA version by analyzing the 10,000 execution results of each test. We label a test as *flaky* within a release if it exhibits both at least one pass and at least one fail: formally, `passed_attempts` > 0 and `failed_attempts` > 0. We then counted flaky tests per version as the number of test cases meeting this criterion.

3.4. Validation with Qiskit Developers

To validate our experimental procedure, we share our experimental setup and preliminary findings with IBM Qiskit developers to obtain practitioner

Table 2: Schema of the per-release execution dataset. Each row corresponds to one test case in one TERRA release.

Attribute	Description
`test_case_name`	The identifier of the individual TERRA test case.
`passed_attempts`	Number of runs in which the test passed.
`failed_attempts`	Number of runs in which the test failed.
`skipped_runs`	Number of skipped runs (e.g., via skip markers or environment constraints); skips were outside our control.
`total_runs`	The total number of executions attempted for the test case; computed as `passed_attempts` + `failed_attempts` + `skipped_runs`; equals 10,000 in all retained rows.
`success_rate`	The fraction of successful executions; computed as `passed_attempts/total_runs`.
`qiskit_version`	The TERRA release tag used for the run.
`python_version`	Python version used (always 3.8).
`rust_version`	Rust toolchain version used to satisfy TERRA build requirements for that release.

feedback on our execution protocol and the interpretation of the detected flaky tests (Qiskit Development Community, 2026a). The developers point out that our earlier configuration used a higher degree of parallelism than is typically used in Qiskit's production CI/CD pipelines. This feedback led us to revise the experimental protocol so that the final reported results better reflect realistic Qiskit testing practice rather than an artificially aggressive stress-testing scenario. After reducing the level of parallelism and re-running the experiments under the revised configuration, the observed level of flakiness decreased. Therefore, the results reported in this paper should be interpreted as flakiness detected under a controlled and practitioner-informed execution protocol, rather than as an exhaustive estimate of all possible flaky behavior that could be exposed under more adversarial or highly parallel settings.

This interaction strengthened this study in two ways. First, it provides external validation from developers directly involved in the Qiskit ecosystem. Second, it clarifies the practical trade-off between maximizing flaky-test dis-

covery and measuring flakiness under realistic operational conditions. More aggressive configurations may expose additional nondeterministic behavior, but they may also overestimate the flakiness experienced by developers in regular CI/CD workflows.

As an additional validation exercise, we apply the revised protocol to a more recent Qiskit version executed with a newer Python environment and identified an additional flaky test that was fixed by Qiskit developers (Qiskit Development Community, 2026b). Although this recent-version result is not included in the main longitudinal analysis, which focuses on the 23 historical releases studied under a consistent Python 3.8 environment, it provides further evidence that the proposed dynamic re-execution approach remains applicable to the current Qiskit ecosystem.

4. Study Results

This section presents the empirical findings of our study. We analyze flakiness across the TERRA releases, address each research question in turn, and then summarize report-level root-cause and fix-pattern observations.

Table 3 summarizes the flakiness count, flakiness rate, and the relative contribution of each release to all observed flaky-test occurrences. Across the 23 releases, we observed 348 flaky-test occurrences, corresponding to 62 unique flaky tests out of 27,026 distinct test cases.

Let us now focus on individual research questions.

4.1. RQ1: How do the count and fraction of flaky tests change between TERRA releases?

Table 3 summarizes the number of flaky tests per release across the 23 TERRA versions studied. We observe substantial variation: the number of flaky tests ranges from 0 (in v.0.25.0, v.0.45.2, v.1.2.3, and v.1.2.4) to 27 (in v.1.0.1 and v.1.0.2), with a mean of 15.1 (SD = 8.6) and a median of 16. Flakiness rates span 0–0.17%, with a mean of 0.08% (SD = 0.05%).

Flakiness is not monotonic over time and shows episodic patterns. The highest absolute counts occur in mid-to-late releases, peaking in the v.0.46.x series (15–25 flaky tests) and the v.1.0.x series (25–27), rather than in the early v.0.25.x releases (0–13). Major-version transitions do not consistently coincide with increases in flakiness: the transition from v.0.46.3 (15) to v.1.0.0 (25) shows a clear jump, but the transition from v.0.25.3 (12) to v.0.45.0 (6) is associated with a decrease, and v.1.0.2 (27) to v.1.1.0 (18) again declines.

Table 3: Test suite size and flakiness across TERRA releases. For each release, we report the total number of test cases, the number of non-skipped tests, the number of flaky tests (i.e., those that both pass and fail across 10,000 runs), the flakiness rate, and the contribution of that release to the global set of unique flaky tests. The flakiness rate is computed as the ratio of flaky test count over total test count. The share of flaky-test occurrences is computed as the ratio of the flaky-test count for a given release over the total number of flaky-test occurrences across all 23 releases. The final two rows aggregate totals across all releases.

Version	Total Test Count	Non-Skipped Tests Count	Flaky Tests Count	Flakiness rate (%)	Share of Flaky-Test Occurrences (%)
0.25.0	21,789	21,445	0	0.00	0.00
0.25.1	21,796	21,303	12	0.06	3.45
0.25.2	21,813	21,320	13	0.06	3.74
0.25.3	21,834	21,339	12	0.06	3.45
0.45.0	21,931	21,447	6	0.03	1.72
0.45.1	21,938	21,455	21	0.10	6.03
0.45.2	21,947	21,464	0	0.00	0.00
0.45.3	21,947	21,464	18	0.08	5.17
0.46.0	21,975	21,491	23	0.11	6.61
0.46.1	21,979	21,494	25	0.12	7.18
0.46.2	21,989	21,504	22	0.10	6.32
0.46.3	21,994	21,509	15	0.07	4.31
1.0.0	16,316	16,122	25	0.16	7.18
1.0.1	16,320	16,126	27	0.17	7.76
1.0.2	16,333	16,139	27	0.17	7.76
1.1.0	17,406	17,208	18	0.10	5.17
1.1.1	17,418	17,220	17	0.10	4.89
1.1.2	17,428	17,229	16	0.09	4.60
1.2.0	17,668	17,468	14	0.08	4.02
1.2.1	17,678	17,474	21	0.12	6.03
1.2.2	17,684	17,480	16	0.09	4.60
1.2.3	17,684	17,480	0	0.00	0.00
1.2.4	17,684	17,480	0	0.00	0.00
Total	452,551	444,661	348	0.08	100.0
Unique	27,026	26,639	62		

Several releases interrupt otherwise active periods with zero observed flaky tests (v.0.45.2 between v.0.45.1 and v.0.45.3; v.0.25.0 at the start of its minor series), suggesting that flakiness can briefly disappear even within a stable

architectural window.

The complete absence of flakiness in the final two releases (v.1.2.3 and v.1.2.4), preceded by a sequence of 14–21 flaky tests in v.1.1.x and v.1.2.0–v.1.2.2, may reflect a more sustained stabilization rather than a transient dip. Overall, the prevalence of quantum test flakiness is episodic: it rises during periods of active development (notably the v.0.46.x and v.1.0.x series), drops sharply in occasional releases, and appears to stabilize in the most recent releases of our study window.

4.2. RQ2: How persistent are individual flaky test cases across TERRA releases?

The answer to the question is given in Table 4. We identified 62 unique flaky tests across the 23 releases. Our dataset shows a markedly different pattern: only 10 tests (16.13%) appear in exactly one release, while the remaining 52 tests (83.87%) recur across multiple releases.

The distribution is concentrated in the mid-range of persistence. The most frequent recurrence span is $k = 5$ releases, accounting for 12 tests (19.35%), followed by $k = 7$ with 11 tests (17.74%) and $k = 6$ with 6 tests (9.68%). Together, these three spans account for 29 tests (46.77%) of all flaky tests, suggesting that a moderate but sustained level of recurrence is the norm rather than the exception in this dataset. Tests with very long persistence ($k \geq 10$) are rare, comprising only 5 tests (8.06%), indicating that indefinite recurrence is uncommon. We will explore patterns of reappearance in Section 4.6.

4.3. RQ3: What is the probability of detecting a flaky test given a specific number of executions?

Here, we answer RQ3 using two approaches: an empirical and a theoretical approach.

4.3.1. Empirical analysis

For each test case, we compute the empirical failure probability

$$\hat{p}_i = \frac{F_i}{N}, \qquad i = 1, \ldots, 62,$$

where F_i is the number of failed runs of test i and $N = 10000$ is the total number of runs of test i.

Table 4: Persistence of flaky tests across TERRA releases. Each row shows the number of tests that were flaky in exactly k TERRA releases within our study window (v.0.25.0–v.1.2.4). The maximum observed persistence was $k = 14$, i.e., no test was flaky in more than 14 releases.

Number of Releases (k)	Flaky Tests Count	Flaky Tests (%)
1	10	16.13
2	3	4.84
3	2	3.23
4	4	6.45
5	12	19.35
6	6	9.68
7	11	17.74
8	6	9.68
9	3	4.84
10	1	1.61
12	1	1.61
13	2	3.23
14	1	1.61
Total	62	100.00

Figure 1 shows the distribution of these estimates.The distribution is bimodal, comprising a dominant cluster of low-probability tests and a smaller cluster of high-probability tests. The majority of flaky tests concentrate at low failure probabilities: 34 of 62 tests (54.84%) have $\hat{p} \leq 10^{-3}$, including 4 tests at the observable floor of $\hat{p} = 10^{-4}$, with a median of $\hat{p} = 9 \times 10^{-4}$. At the same time, the upper tail is non-negligible: 13 tests exhibit $\hat{p} \in (0.1, 1)$, and 3 tests fail in nearly every run ($\hat{p} \geq 0.99$), with the most extreme case reaching $\hat{p} = 1.0$.

It is easy to miss test flakiness when $\hat{p}$ is low (e.g. $\hat{p} = 10^{-4}$) under small rerun budgets. Let us perform a theoretical analysis to quantify the hardness.

4.3.2. Theoretical analysis

How many repetitions are needed to confidently detect a flaky test? Assuming independent and identically distributed (IID) test outcomes, failures follow a binomial distribution. To bound the true failure probability p given

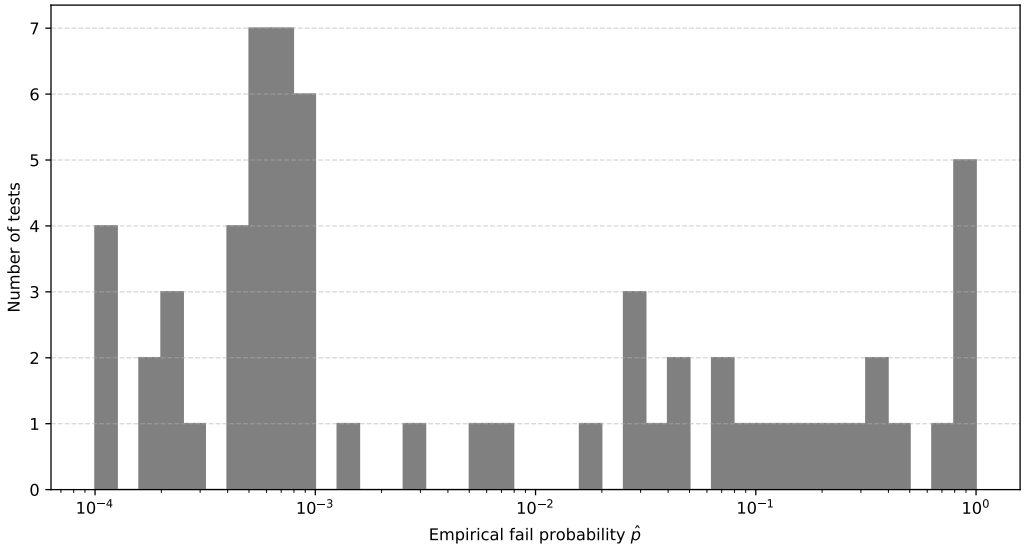


Figure 1: Distribution of empirical failure probabilities $\hat{p}$ for all flaky tests in our dataset (62 unique tests, aggregated across releases). The distribution is fat-tailed and exhibits two distinct modes: a dominant cluster of low-probability tests concentrated between $\hat{p} = 10^{-4}$ and 10^{-3} (median $\hat{p} \approx 9 \times 10^{-4}$; 4 tests at the observable floor of $\hat{p} = 10^{-4}$), and a smaller cluster of high-probability tests near $\hat{p} \in [10^{-1}, 1]$, with the most extreme case reaching $\hat{p} = 1.0$.

an empirical estimate $\hat{p}$ at a chosen confidence level, we use the Wilson confidence interval for binomial proportions (Wilson, 1927); see Brown et al. (2001) for a review.[4]

Figure 2 shows how the Wilson interval varies with $\hat{p}$ at different confidence levels γ. As $\hat{p}$ decreases, the interval widens.

We next connect these bounds to a conservative rerun budget for flakiness detection. Let X be the number of observed failures after n executions, so

[4]The IID assumption provides a baseline detectability model rather than a guarantee for every execution environment. Repeated HPC executions may be correlated or overdispersed because of shared machine state, filesystem cache warm-up, CPU frequency scaling, scheduler behavior, dependency resolution, or job contention. Accordingly, the Wilson intervals and rerun-budget estimates should be read as idealized Bernoulli-trial calculations, especially for very small empirical probabilities. Even under this optimistic model, however, tests with $\hat{p} \approx 10^{-4}$ require rerun budgets far beyond typical CI practice. Relaxing IID would not make such failures easier to detect; it would mainly motivate more conservative or environment-aware uncertainty estimates.

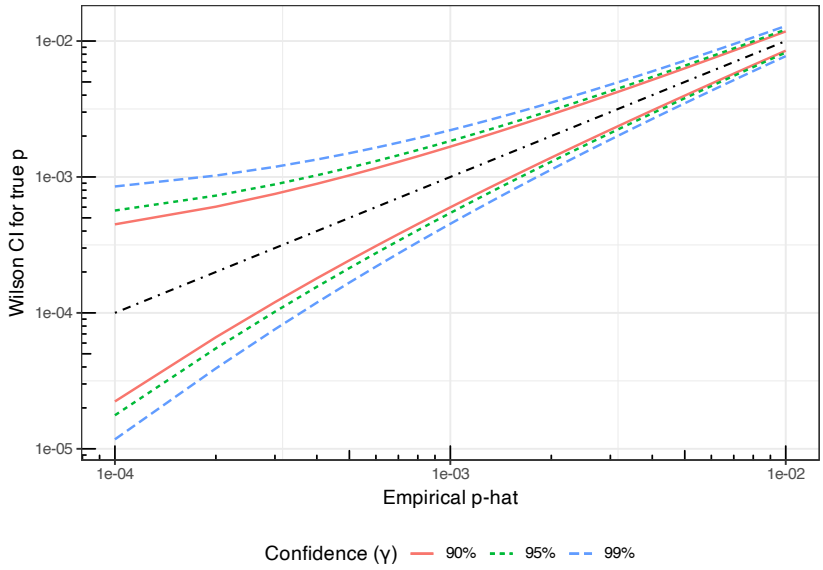


Figure 2: Wilson confidence intervals for the true failure probability p as a function of the empirical estimate $\hat{p}$ when $N = 10000$. For each confidence level $\gamma \in \{90\%, 95\%, 99\%\}$, the upper and lower bounds are shown as paired lines with matching color and line type. The reference line $p = \hat{p}$ is shown with a dot-dash pattern.

that $X \sim \text{Binomial}(n, p)$. In general, the probability of observing exactly k failures is

$$\Pr(X = k) = \binom{n}{k} p^k (1-p)^{n-k}, \qquad k = 0, 1, \ldots, n.$$

Therefore, the probability of detecting flakiness by observing at least one failure is

$$\Pr(X \geq 1) = 1 - \Pr(X = 0) = 1 - \binom{n}{0} p^0 (1-p)^n = 1 - (1-p)^n. \tag{1}$$

Given a target detection probability $q \in (0, 1)$, we require

$$1 - (1-p)^n \geq q \quad \Leftrightarrow \quad (1-p)^n \leq 1 - q \quad \Rightarrow \quad n \geq \frac{\ln(1-q)}{\ln(1-p)}, \tag{2}$$

as $\ln(1-p) < 0$.

In practice, p is unknown. Let $[L, U]$ be the Wilson confidence interval for p with nominal coverage γ, computed from N runs, and let $L = L_\gamma(\hat{p}, N)$

denote its lower bound. Using L as a conservative lower bound on the unknown p, and noting that Equation (1) is monotonically increasing in p, the worst case occurs at $p = L$. Therefore, a conservative rerun budget that targets detection probability at least q is

$$n_{q,\gamma}(\hat{p}, N) = \left\lceil \frac{\ln(1-q)}{\ln\left[1 - L_\gamma(\hat{p}, N)\right]} \right\rceil,$$

where $\lceil \cdot \rceil$ denotes the ceiling function.

Figure 3 plots $n_{q,\gamma}(\hat{p}, N)$ versus $\hat{p}$ (for fixed N and q), illustrating how required repetitions grow rapidly as $\hat{p}$ decreases and increase further with higher confidence γ. For example, with $q = 0.95$ and $\gamma = 0.95$, when $N = 10000$ and $\hat{p} = 10^{-4}$, then the 95% Wilson lower bound is $L_{0.95} \approx 1.77 \times 10^{-5}$. Plugging these values into Equation (2), yields $n_{0.95,0.95}(0.0001, 10000) \approx 1.69 \times 10^5$ repetitions. When $\hat{p} = 0.01$ and $L \approx 8.23 \times 10^{-3}$, this reduces to $n_{0.95,0.95}(0.01, 10000) \approx 363$ repetitions.

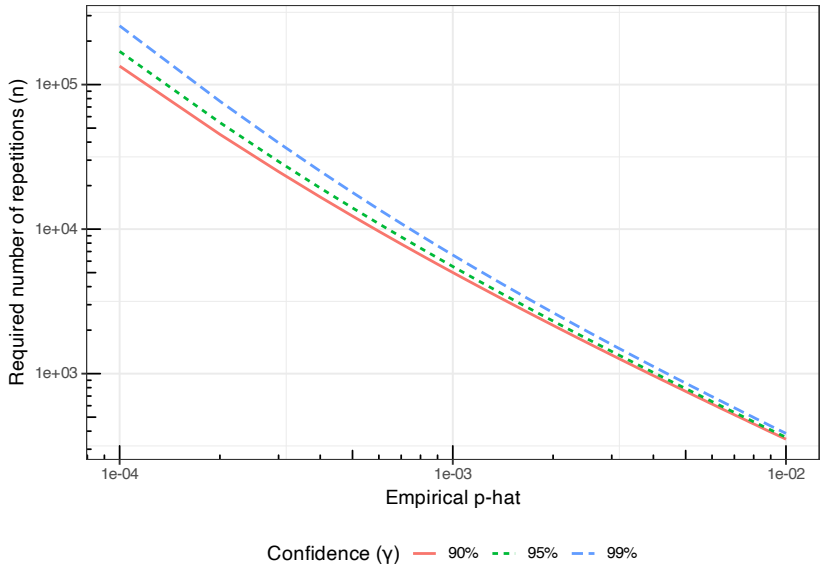


Figure 3: Conservative rerun budget for flakiness detection using Wilson lower confidence bounds. The x-axis shows the empirical estimate $\hat{p}$ from $N = 10000$ observed runs, and the y-axis shows the required number of repetitions $n_{q,\gamma}(\hat{p}, N)$ with $q = 95\%$ and $\gamma \in \{90\%, 95\%, 99\%\}$. Higher confidence levels require more repetitions.

To make these implications concrete for practitioners, consider typical per-commit CI rerun budgets of $n = 50$, 100, and 1,000. Using $\Pr(X \geq 1) =$

$1-(1-p)^n$, a test with true failure probability $p = 10^{-3}$ is detected with probability approximately 4.9%, 9.5%, and 63.2%, respectively, while a test with $p = 10^{-4}$ is detected with probability approximately 0.5%, 1.0%, and 9.5%. In other words, even a CI process that reruns a test one thousand times has less than a 10% chance of surfacing a flake whose true failure probability is on the order of 10^{-4}, which is consistent with our observation that many quantum flaky tests recur across releases without being caught. Therefore, our results do not imply that every test should be executed thousands of times for every pull request. Instead, they motivate a tiered execution strategy: small rerun budgets for routine per-commit validation, intermediate budgets for nightly testing, and larger rerun budgets for pre-release validation or historically flaky tests. Without such tiering, the low-probability mode identified in Figure 1 will remain difficult to detect.

4.4. RQ4: How are failure intensities distributed across releases?

To characterize failure intensity within a release, we grouped empirical failure counts (out of 10,000 runs) into four bins: $(0, 10]$, $(10, 100]$, $(100, 1000]$, and $(1000, 10000]$. As shown in Figure 4, the distribution is dominated by the lowest bin, where $\hat{p} \leq 10^{-3}$. High-frequency flaky tests ($\hat{p} > 0.1$) are rare but present. Notably, tests in the $(100, 1000]$ bin appear relatively late, starting with v.0.45.1.

4.5. RQ5: How are flaky tests distributed across TERRA subcomponents?

To understand which parts of TERRA are most affected by nondeterminism, we mapped each test to its subcomponent. Subcomponent descriptions are provided in Appendix A. Figures 5 and 6 show that flakiness is highly nonuniform across the subcomponents.

Figure 5 indicates that the `quantum_info` and `synthesis` subcomponents have the highest number of distinct flaky tests (13), followed by `scheduler` (10) and `pulse` (9) . In contrast, subcomponents such as `algorithms` and `text_examples` each contain only one flaky test.

Figure 6 provides a per-release breakdown, highlighting further variation across subcomponents. For example, the `quantum_info` subcomponent shows the highest median number of distinct flaky tests per release(median ≈ 12, $n = 6$ releases), followed closely by `synthesis` (median ≈ 11, $n = 7$). The `scheduler` component, while present in 10 releases, exhibits the widest spread—ranging from 1 to 10 flaky tests per release—suggesting high sensitivity to release-to-release changes. The `pulse` and `qpy` subcomponents

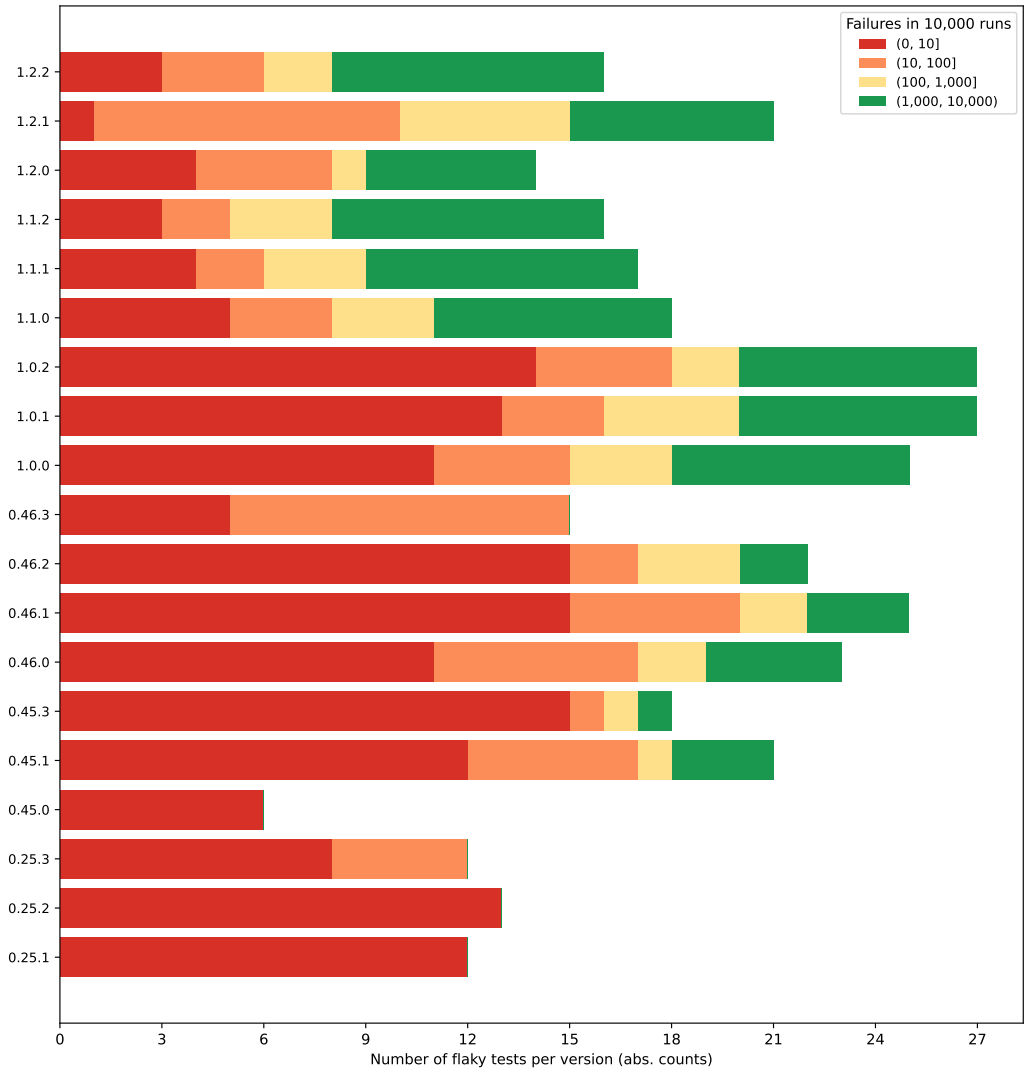


Figure 4: Distribution of failure frequencies for flaky tests across TERRA releases. Each horizontal bar partitions a release's flaky tests into four bins based on the number of failures observed in 10,000 executions.

appear in the most releases ($n = 14$ each), yet maintain a narrower range of flaky test counts, with `pulse` having a median of 5 and `qpy` a median of 3.

Overall, these results suggest that the flakiness in TERRA is concentrated in a few key subcomponents, such as `quantum_info` and `synthesis`. The wide variation in flaky test frequency across releases also indicates that some

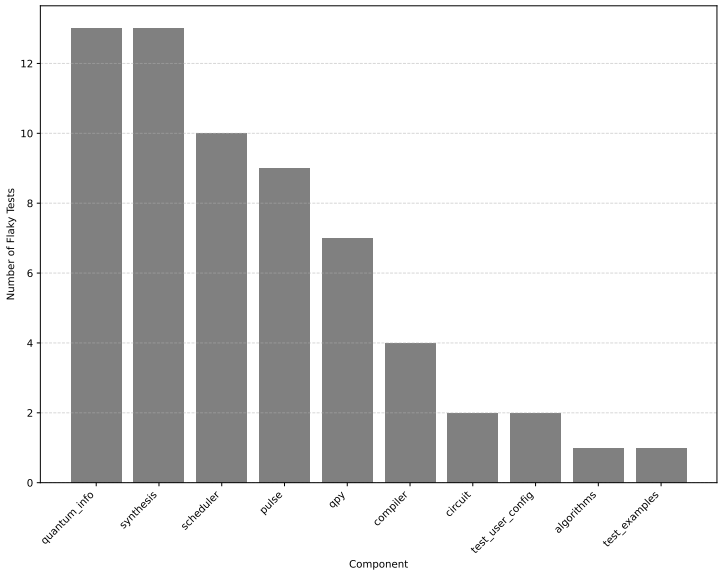


Figure 5: Total number of distinct flaky tests by TERRA subcomponent, aggregated across all releases. Subcomponents are sorted in descending order.

components are more sensitive to changes in the codebase or external dependencies. Understanding this distribution may help prioritize stabilization efforts and guide targeted improvements in test reliability.

4.6. RQ6: How do flaky tests emerge, dissipate, or recur across the sequence of TERRA releases?

We analyzed time-series patterns of flaky appearances across releases and identified three characteristic temporal profiles.

Rarely Flaky Tests flaky in $\leq 15\%$ of their active releases. This subtype accounts for 14 tests (22.58%). These failures are rare; examples are presented in Figure 7.

Persistently Flaky Tests flaky in $\geq 70\%$ of the releases in which they appear. Only 7 tests (11.29%) fall into this group. They frequently fail but with variable intensity, often associated with unstable transformations in the transpiler. They often fail with fluctuating intensity; examples are presented in Figure 8.

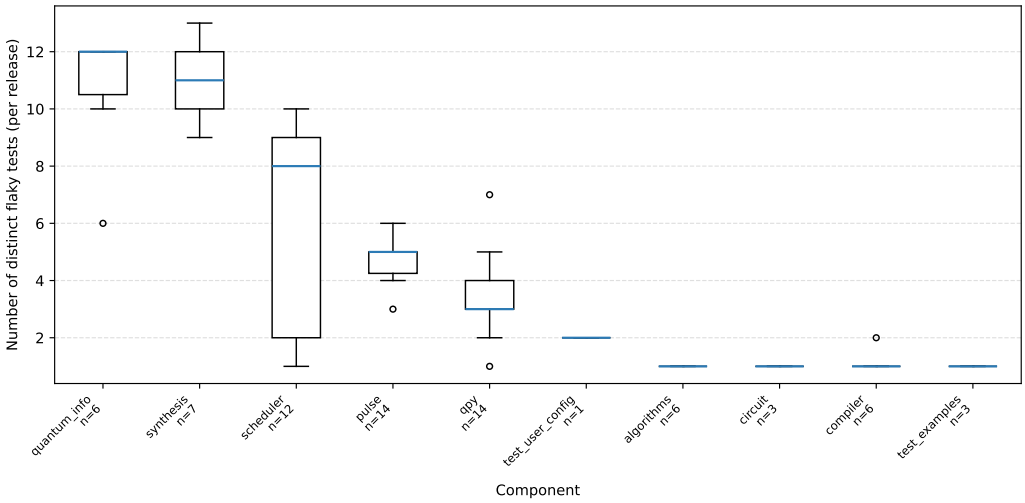


Figure 6: Distribution of flaky tests across TERRA subcomponents over all analyzed releases. For each subcomponent, the boxplot summarizes the per-release count of distinct flaky test cases, and subcomponents are ordered as in Figure 5, where n indicates the number of releases in which flaky tests are observed for that subcomponent.

Intermittently Flaky Tests that alternate between flaky and stable states across multiple releases. This subtype includes 41 tests (66.13%) and is characterized by an intermittent, back-and-forth pattern where each flaky period is brief. Examples are presented in Figure 9.

The streak-length distribution in Figure 10 shows that maximum consecutive-flakiness streaks span the full observed range of 1–9 releases and are broadly distributed across it. The most frequent maximum streak is $m = 3$ (13 tests), closely followed by isolated single-release flakiness ($m = 1$, 12 tests) and by long streaks of $m = 7$ (10 tests). Only 12 of 62 tests (19.35%) have a maximum consecutive flaky-release streak of one release, although some of these tests may recur in multiple non-consecutive releases. We also observe 18 tests (29.03%) sustain streaks of 7 or more consecutive releases. Flakiness in TERRA is therefore neither purely fragmented nor uniformly persistent: once a test becomes flaky, it frequently—though not always—remains flaky across several consecutive releases.

4.7. Root-Cause Categories and Fix Patterns

To understand why the flaky tests in our dataset fail and how they were ultimately addressed in Qiskit, we manually inspected each of the 62 unique

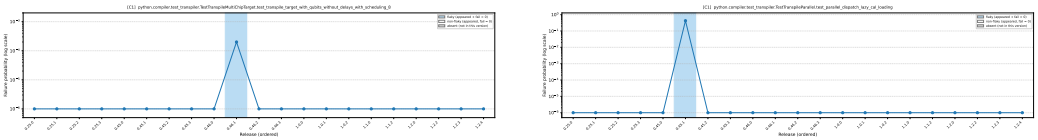


Figure 7: Rarely Flaky pattern. The temporal pattern in the right panel does not imply a sequence of flaky–fixed–reintroduced states. Rather, the underlying failure event is extremely rare, so the true failure probability is low and may not be consistently observed (see Section 4.3.2). Because a log scale cannot display zero, we use 10^{-6} as a placeholder for zero observed failures. The lowest observable non-zero rate is 10^{-4}, corresponding to one failure in 10,000 runs.

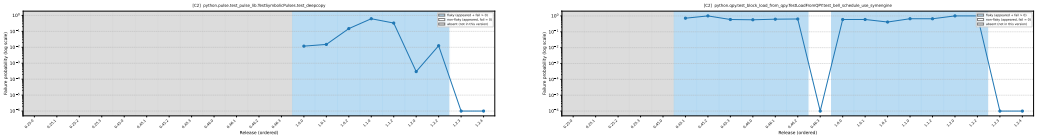


Figure 8: Persistently Flaky pattern. The gray background marks releases in which the test case did not exist or was skipped. In the left panel, the test is present across several releases and consistently flaky until removal. In the right panel, the test is introduced later, remains consistently flaky, and appears to be fixed only in the final two releases. As log scales cannot display zero, we use 10^{-6} for zero observed failures; the lowest observable non-zero rate is 10^{-4}.

flaky tests and assigned both a root-cause category and a fix-pattern category. We use the taxonomy for quantum flaky tests proposed by Sivaloganathan et al. (2026), which builds on the classical flaky-test taxonomies of Luo et al. (2014); Lam et al. (2019a). For clarity, we group low-count categories into an *Others* bucket. The full per-test annotations, including the specific root-cause and fix subcategories, are provided in the supplementary spreadsheet `qiskit_flaky_tests_root_cause_fix_labels.xlsx`, released as part of the public dataset at `https://zenodo.org/records/20500432`.

4.7.1. Verification Method

For each unique flaky test, two authors independently labeled the root-cause and fix-pattern. Each reviewer examined the same evidence sources without seeing the other's labels: (i) the test source at the last release in which the test was flaky, (ii) the failing assertion and any reproducible traceback, (iii) the implementation of the exercised component, such as a transpiler pass, a synthesis routine, or the symbolic-pulse backend, and (iv) the version-control history between the last flaky release and the first stable or

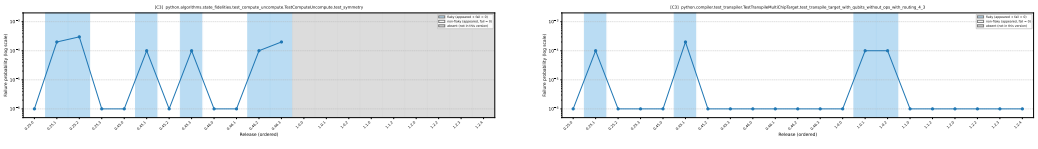


Figure 9: Intermittently Flaky pattern. These examples show test cases that intermittently alternate between flaky and stable states. We do not speculate on the underlying causes, but this pattern contrasts with consistently failing tests such as those in Figure 8. As in Figures 7 and 8, 10^{-6} denotes zero observed failures, and the smallest observable non-zero rate is 10^{-4}.

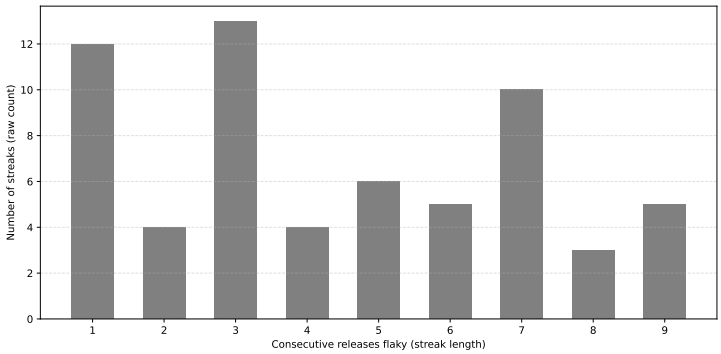


Figure 10: Longest consecutive flaky-release streak per test across TERRA releases. Bars show the number of flaky tests whose maximum streak length is m, where a streak is a maximal run of m consecutive releases in which a test is flaky. Streak lengths span the full observed range (1–9), and the most frequent maximum streak length is $m = 3$ (13 tests), with $m = 1$ (12 tests) and $m = 7$ (10 tests) also common.

removal release, including commits, pull requests, and linked GitHub issues when available.

The reviewers then reconciled their labels jointly. Agreed labels were retained, and disagreements were resolved by re-examining the evidence until consensus was reached. We also used a large language model Claude Opus 4.7 (Anthropic (2025)) to cross-check the reviewers' notes against the same evidence and flag possible inconsistencies; however, the final labels were always assigned by the reviewers. The reconciled labels form the basis for Table 5.

4.7.2. Root-Cause Category

We classify root causes using the quantum flaky-test taxonomy of Sivaloganathan et al. (2026), which defines nine root-cause categories. Four cat-

Table 5: Count of root-cause categories and fix patterns based on manual verification.

	Fix category						
Root-cause category	Remove Functionality	Rewrite Component	Fix Seed	Change Test Env.	Remove Test Case	Grand Total	Percentage
Floating Point Operations	6	24	0	0	0	30	48.4%
Multi-threading	0	0	0	0	1	1	1.6%
Randomness	0	0	3	0	1	4	6.5%
Others	23	1	0	3	0	27	43.5%
Grand Total	29	25	3	3	2	62	100%
Percentage	46.8%	40.3%	4.8%	4.8%	3.2%	100%	

egories occur in our data: *Floating Point Operations*, *Randomness*, *Multi-Threading*, and *Others*. The remaining five categories (*Software Environment*, *Visualization*, *Unhandled Exception*, *Network*, and *Unordered Collection*) did not match any of our observed flaky tests and are therefore omitted from the discussion below.

Floating Point Operations. This category covers flaky tests caused by numerical precision issues, such as round-off errors. It is the dominant root cause in our dataset, accounting for 30 of 62 tests (48.4%). The recurring pattern is a strict equality or tight-tolerance assertion applied to values computed through quantum operators or transformations, where small differences near machine precision can change the test outcome. A representative example is the family of tests in `quantum_info` (later moved to `synthesis`) using `TwoQubitWeylDecomposition`. In these tests, a round-trip through the Weyl/KAK decomposition produced matrix entries that differ by approximately 1.75×10^{-16} between runs and violated the assertion. The underlying issue (Qiskit #11676) was ultimately addressed by reimplementing the decomposition kernel in Rust (PR #11946). The much higher share of floating-point failures in our dynamic dataset than the 9.6% reported from issue and pull-request text in Sivaloganathan et al. (2026) is consistent with the expectation that dynamic re-execution surfaces precision-driven failures that may never generate an explicit issue report.

Randomness. This category covers flaky tests caused by pseudo-random choices that vary across runs when no fixed seed is used. It accounts for 4 of 62 tests (6.5%). A representative case is `test_transpile_target_with_qubits_without_delays_with_scheduling_8` in the transpiler subcomponent, where unseeded layout and routing search on a disconnected multi-chip target produces nondeterministic outcomes. The fix was to set `seed_transpiler`. Randomness is the dominant category (19.2%) in the issue and pull-request data of Sivaloganathan et al. (2026), but it represents only a

small share of our dynamic observed failures. One plausible explanation is that our analysis uses code snapshots from shipped Qiskit releases, so seed-controllable randomness may have been identified and fixed during release preparation, leaving fewer such failures to be observed through re-execution.

Multi-Threading. This category covers flaky tests caused by concurrency, scheduling, and overload effects. In our dataset, only 1 of 62 tests (1.6%) fall into this category, substantially lower than the 13.7% reported in the issue and pull-request data of Sivaloganathan et al. (2026).

Others. The remaining 27 tests (43.5%) fall into the *Others* bucket. These tests correspond to low-count or ambiguous mechanisms that did not warrant separate categories in the summary table. The detailed per-test labels are provided in the supplementary spreadsheet.

4.7.3. Fix Category

We also classify how each flaky test was ultimately addressed in the Qiskit codebase. The observed fix patterns only partially overlap with the taxonomy of Sivaloganathan et al. (2026): *Fix Seed* maps directly to controlling randomness, and *Change Test Environment* is closely related to altering the software or test environment. The remaining patterns (*Remove Functionality*, *Rewrite Component*, and *Remove Test Case*) emerged from our manual inspection of Qiskit's version history. We define these categories below.

Remove Functionality. This category captures cases where the flaky test disappeared because the functionality it exercised was removed, deprecated, or subsumed by another implementation. It is the most common fix pattern in our data, accounting for 29 of 62 tests (46.8%). In these cases, the disappearance of flakiness does not necessarily mean that the underlying nondeterminism was diagnosed and repaired directly; rather, the affected functionality was no longer part of the maintained code path. Thus, some flaky tests appear to have been resolved as a by-product of broader codebase evolution rather than through targeted flaky-test repair.

Rewrite Component. This category captures cases where the exercised implementation was substantially rewritten while preserving the intended functionality. It is the second most common fix pattern, accounting for 25 of 62 tests (42.4%). Several floating-point-related failures disappeared after numerical or synthesis components were reimplemented, including cases where

Python implementations were replaced by Rust-based kernels. For example, the `TwoQubitWeylDecomposition` failures discussed above were addressed by reimplementing the decomposition kernel in Rust. We do not claim that the language change alone explains the disappearance of flakiness; rather, the rewrite changed the numerical implementation sufficiently that the previously observed precision-sensitive failures no longer manifested.

Fix Seed. This category covers cases where nondeterminism was addressed by controlling a random source, typically by setting or propagating a deterministic seed. It accounts for 3 of 62 tests (4.8%) and corresponds closely to the *Randomness* root-cause category. For example, tests involving stochastic transpiler behavior can fail when layout, routing, or synthesis choices vary across executions while the assertion implicitly assumes one outcome. Setting the seed makes the test deterministic by forcing the same randomized choice across runs.

Change Test Environment. This category captures cases where flakiness was addressed by changing the test environment rather than the tested functionality. It accounts for 3 of 62 tests (4.8%). Such changes may involve test configuration, dependencies, execution conditions, or other environmental assumptions that affect determinism. These cases are close to the *Alter Software Environment* fix pattern in prior taxonomies; we use the narrower label *Change Test Environment* to emphasize that the observed change affected the testing setup rather than necessarily changing the production implementation.

Remove Test Case. This category captures cases where the flaky test itself was removed while the broader functionality was not clearly removed at the same time. It accounts for 2 of 62 tests (3.2%). Unlike *Remove Functionality*, this pattern indicates that the unstable test was eliminated from the suite even though the exercised functionality may have remained. As with functionality removal, this does not necessarily mean that the root cause of nondeterminism was repaired.

Overall, the fix-pattern distribution shows that most flaky tests in our dataset were not repaired through small, localized changes such as setting a seed or adjusting the test environment. Instead, 54 of 62 tests (87.1%) disappeared after either functionality removal or substantial component rewrites. This finding should be interpreted cautiously: version-control history reveals what changed, but not always why. A test not being fixed directly should not

be read as evidence that developers ignored or deprioritized it. In some cases, the affected component may have already been slated for removal, deprecation, or rewrite, making a targeted flaky-test repair economically unjustified. Leaving the test unresolved until the broader component change landed may therefore have been a deliberate maintenance trade-off rather than a lack of attention to test quality. Because we cannot generally observe developers' internal rationale from repository history alone, we treat this explanation as an interpretation rather than a directly verified cause. Nevertheless, the distribution suggests that flaky-test disappearance in our dataset is closely tied to broader software maintenance and architectural evolution, not only to targeted flaky-test debugging.

5. Discussion

The preceding section presented the empirical findings for each research question. We now synthesize these findings and discuss their implications for testing practice, maintenance, and future tool development in quantum software.

5.1. Cross-Cutting Observations

Considering the six RQs jointly reveals a "rare-but-recurrent flakiness" pattern. Combining the observations in Tables 3 and 4 and Figure 1, we observe the coexistence of three properties that, taken together, define a particularly difficult-to-handle class of flakiness in TERRA: (i) per-release flakiness rates are low, ranging from 0% to 0.17%; (ii) most flaky tests recur across multiple releases, with 52 of 62 unique flaky tests (83.87%) appearing in more than one release; and (iii) the empirical failure probabilities of many flaky tests are very small, with a median $\hat{p} \approx 9 \times 10^{-4}$ and 34 tests (54.8%) at or below 10^{-3}. A test that fails roughly once in every thousand or ten thousand executions is unlikely to be observed under typical CI rerun budgets, yet such a test may continue to appear across releases. We refer to this observation as "rare-but-recurrent flakiness": flaky behavior that is infrequent enough to evade ordinary CI rerun budgets, yet recurrent enough to remain relevant across releases.

5.2. Implications for Quantum Software Testing Practice

The patterns above translate into several practical recommendations for teams developing or maintaining quantum software stacks of TERRA's scale and complexity.

Maintain cross-release flaky-test histories. Because most flaky tests in our dataset recur across multiple releases, release-local quarantine policies may lose information needed for diagnosis. A test that appears stable in one release may still have a history of intermittent flakiness. We therefore recommend maintaining a persistent registry of flaky test identifiers, together with their empirical failure rates, affected releases, suspected cause category, and applied fix pattern where available. Such a registry would make recurrence patterns explicit during triage and would allow analyses like those in Section 4.6 to be performed continuously rather than retrospectively.

Triage tests by temporal pattern and failure intensity. The three temporal modes identified in RQ6 suggest different response strategies. Persistently Flaky tests should be prioritized because they represent recurring reliability signals rather than isolated anomalies. Intermittently Flaky tests should be treated as recurrence-prone and may benefit from environment lock-in, dependency pinning, seed control, and closer inspection of tolerance thresholds or nondeterministic test logic. Rarely Flaky tests should be interpreted cautiously: dynamic outcomes alone may be insufficient to identify the mechanism, and tests that intentionally exercise stochastic behavior may require statistical assertions rather than deterministic pass/fail thresholds.

Treat numerical tolerance and seed control as first-class testing concerns. Our root-cause analysis (Table 5) shows that floating-point operations are the dominant mechanism, accounting for 30 of 62 flaky tests (48.4%), typically arising from strict-equality or tight-tolerance assertions on values computed through quantum operators and transformations. This suggests that tolerance thresholds should be chosen deliberately and documented as explicit test-design decisions rather than left implicit. Although randomness accounts for only 4 tests (6.5%) in our dataset, 3 such cases were resolved by seed control, indicating that propagating deterministic seeds through layout, routing, and synthesis remains a cheap and effective safeguard for the stochastic components of TERRA.

Avoid interpreting zero observed failures as proof of resolution. Table 3 shows zero observed flaky tests in v.0.25.0, v.0.45.2, v.1.2.3, and v.1.2.4. While the absence of observed flakiness in the two most recent releases may reflect genuine stabilization, our probabilistic analysis cautions against interpreting zero observations as proof that flakiness has been eliminated. A test with a very low failure probability can produce zero failures in 10,000 executions with

non-negligible probability. Thus, zero observed failures should be read as "no flakiness detected at this budget," not necessarily as "flakiness eliminated."

5.3. Implications for Tool and Research Development

Beyond immediate testing practice, our findings inform several directions for tooling and research targeted at quantum software reliability.

Dynamic ground truth is essential for ML-based detection. Existing machine-learning approaches to quantum flaky-test detection rely largely on incident-reported or pre-existing labeled cases. Our findings suggest that such labels may under-represent low-probability flaky tests, which are unlikely to be noticed under ordinary CI budgets. The dynamic execution data produced in this study provides a complementary source of supervision and evaluation: future predictors can be assessed not only by binary flaky/non-flaky labels, but also by empirical failure probability, recurrence across releases, subcomponent membership, and historical fix patterns.

Release-level dynamic evidence captures persistent flaky tests. Our dynamic analysis characterizes flaky tests that are present at release points, and therefore reflects the subset of flaky behavior that survived the development and integration process. This perspective is complementary to issue- and pull-request-based taxonomies, but it should not be interpreted as a complete picture of all flaky tests encountered during development. Some flaky tests may be easier to detect, reproduce, or fix during regular development and CI, and may therefore be removed before a release is shipped. Conversely, the flaky tests observed in our study may represent more persistent or harder-to-eliminate cases.

This distinction matters when comparing root-cause distributions across studies. Floating-point precision failures account for 48.4% of the release-level flaky tests in our dataset, while randomness and multi-threading are less frequent. However, this does not imply that precision issues dominate the full population of quantum flaky tests across the development lifecycle. Rather, our results show that release-level dynamic studies and report-level studies can expose different slices of the flaky-test population. Characterizing flaky tests that arise and are resolved between releases remains an important direction for future work.

Replication beyond one quantum framework remains necessary. Finally, the patterns reported here are derived from one quantum software framework, Qiskit TERRA, under a specific Linux-based containerized test environment. Although TERRA is large, mature, and widely used, replication across other quantum stacks such as Cirq, PennyLane, and Q#, as well as across hardware-backed execution environments, is needed before these patterns can be generalized to quantum software as a whole. Our methodology and dataset are intended to support such replication.

5.4. Summary of Practical Recommendations

Based on the results and discussion above, we recommend that quantum software projects: (1) maintain cross-release histories of flaky tests; (2) use tiered rerun budgets for routine, nightly, pre-release, and historically flaky tests; (3) prioritize triage using both empirical failure probability and temporal recurrence; (4) treat seed control, numerical tolerance, and stochastic assertions as explicit test-design decisions; (5) record suspected root causes and applied fix patterns alongside flaky-test reports; and (6) validate future static or ML-based flaky-test detectors against dynamic execution-level evidence.

Together, these recommendations emphasize that flaky-test management in quantum software should be both probabilistic and historical. Low observed flakiness rates do not necessarily imply low engineering risk when failures are rare, recurrent, and difficult to detect under ordinary CI budgets.

6. Threats to Validity

We discuss threats to the validity of our case study following the classifications of Wohlin et al. (2012); Yin (2009).

6.1. Internal

Our methodology is derived from the classical experimental methodology of Flake Flagger (Alshammari et al., 2021): a high-budget rerun protocol (10,000 executions per test per release) in a controlled environment without injecting additional non-determinism. While this approach provides robust empirical data, it also introduces several risks.

First, repeated executions on an HPC cluster may not be independent or identically distributed due to factors like CPU frequency scaling, file system

cache warm-up, and job contention, which can cause empirical failure rates to drift within a batch.

Second, unlike Lam et al. (2019b); Bell et al. (2018), we do not reorder tests, perturb timing, or inject failures. As a result, we may under-detect order-dependent and timing-sensitive flaky tests.

Third, although we pin Python (3.8) and specific versions of the Qiskit and Rust toolchains, some transitive dependencies are resolved at runtime. Changes in mirror or point-release updates can introduce subtle environment differences over time. To mitigate this, we containerize each version and fix the test orchestrator configuration. However, full immutability is difficult to guarantee without fully offline and locked dependencies.

Fourth, the root-cause and fix-pattern labels rely on manual interpretation of test code, failure traces, implementation changes, and version-control history. This creates a risk of mislabeling, especially when the repository history does not explicitly discuss flakiness or when several changes occur between the last flaky release and the first stable release. To mitigate this risk, at least two authors independently labeled each test using the same evidence sources, reconciled disagreements through joint re-examination, and involved additional authors when the evidence remained ambiguous. Nevertheless, some labels, particularly within the *Others* bucket and broader fix categories, should be interpreted as best-effort classifications rather than definitive causal diagnoses.

6.2. External

Our results are based on the TERRA component running on Linux within Singularity containers on Intel-based HPC nodes. Hardware characteristics (e.g., ISA, cache hierarchy), operating system behavior, container runtime, and CI topology all influence timing and scheduling. Other environments (e.g., macOS/Windows, AMD CPUs, cloud-based CI) may exhibit different flakiness patterns.

We restricted the release range to v.0.25.0–v.1.2.4 due to toolchain constraints. Earlier or later versions, or other quantum computing frameworks, may behave differently (Zhang et al., 2023). Therefore, generalization beyond Qiskit, beyond Python, or to hardware backends (as opposed to simulators) should be approached with caution.

As in many software engineering studies, the variability of real-world environments limits generalizability (Wieringa and Daneva, 2015). Nonethe-

less, our empirical methodology is reproducible and can be applied to other frameworks. We encourage the community to replicate and extend our work.

7. Conclusions and Future Work

This paper presents the first dynamic study of flaky tests in quantum software. By re-executing the Qiskit TERRA test suite 10,000 times across 23 releases, we provide a comprehensive empirical characterization of nondeterministic behavior in a quantum software testing framework.

Our findings show that quantum test flakiness in Qiskit TERRA is rare at the aggregate level, but difficult to manage in practice. Many flaky tests cluster at empirical failure probabilities of 10^{-3} or below, making them unlikely to be detected under typical CI rerun budgets, while many also recur across multiple releases. Many flaky tests exhibit very low empirical failure probabilities, making them unlikely to be detected under typical CI rerun budgets, while many also recur across multiple releases. These results suggest that quantum flaky-test management should be probabilistic, historical, and budget-aware, rather than relying only on release-local observations or small rerun counts.

We also release a publicly available dataset containing per-test execution results for all 23 releases, providing dynamic ground truth for future work in quantum software testing, debugging, and machine-learning prediction.

Future work includes (i) extending our analysis to other quantum software frameworks, (ii) exploring more efficient or adaptive detection strategies, and (iii) developing machine-learning models that leverage both static and dynamic signals for more robust detection and prediction of quantum flaky tests.

Acknowledgement

This work was partially supported by the Natural Sciences and Engineering Research Council of Canada (grant # RGPIN-2022-03886) and the Strategic Awards for Research Transitions (START) award from University of Maryland, Baltimore County. The authors thank the Digital Research Alliance of Canada for providing computational resources. We are also grateful to Dr. Jake Lishman of the Qiskit development team for reviewing our experimental setup and preliminary findings, and for helping us validate and refine the study protocol.

References

Alshammari, A., Morris, C., Hilton, M., Bell, J., 2021. Flakeflagger: Predicting flakiness without rerunning tests, in: Proceedings of the 43rd International Conference on Software Engineering (ICSE 2021) – Companion Volume, IEEE. p. 187. doi:10.1109/ICSE-COMPANION52605.2021.00081.

Anthropic, 2025. Claude opus 4.7. https://www.anthropic.com/claude. Large language model.

Bell, J., Legunsen, O., Hilton, M., Eloussi, L., Yung, T., Marinov, D., 2018. Deflaker: Automatically detecting flaky tests, in: Proceedings of the 40th International Conference on Software Engineering (ICSE '18), ACM. pp. 433–444. doi:10.1145/3180155.3180164.

Brown, L.D., Cai, T.T., DasGupta, A., 2001. Interval estimation for a binomial proportion. Statistical science 16, 101–133.

Dutta, S., Shi, A., Choudhary, R., Zhang, Z., Jain, A., Misailovic, S., 2020. Detecting flaky tests in probabilistic and machine learning applications, in: Proceedings of the 29th ACM SIGSOFT international symposium on software testing and analysis, pp. 211–224.

Eck, M., Palomba, F., Castelluccio, M., Bacchelli, A., 2019. Understanding flaky tests: The developer's perspective, in: Proceedings of the 27th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '19), ACM. pp. 830–840.

Gruber, M., Fraser, G., 2022. A survey on how test flakiness affects developers and what support they need to address it, in: Proceedings of 2022 IEEE Conference on Software Testing, Verification and Validation (ICST), IEEE. pp. 82–92.

Gruber, M., Lukasczyk, S., Kroiß, F., Fraser, G., 2021. An empirical study of flaky tests in Python, in: 2021 14th IEEE Conference on Software Testing, Verification and Validation (ICST), IEEE. pp. 148–158.

Javadi-Abhari, A., Treinish, M., Krsulich, K., Wood, C.J., Lishman, J., Gacon, J., Martiel, S., Nation, P.D., Bishop, L.S., Cross, A.W., Johnson, B.R., Gambetta, J.M., 2024. Quantum computing with Qiskit. doi:10.48550/arXiv.2405.08810, arXiv:2405.08810.

Kaur, K., Kim, D., Jamshidi, A., Zhang, L., 2025. Identifying flaky tests in quantum code: A machine learning approach, in: Proceedings of the 8th Workshop on Validation, Analysis and Evolution of Software Tests (VST 2025). doi:10.48550/arXiv.2502.04471.

Khan, A.A., Ye, B., Akbar, M.A., Khan, J.A., Mougouei, D., Ma, X., 2025. Mining q&a platforms for empirical evidence on quantum software programming. arXiv preprint arXiv:2503.05240 .

Kurtzer, G.M., Sochat, V., Bauer, M.W., 2017. Singularity: Scientific containers for mobility of compute. PloS one 12, e0177459.

Lam, W., Godefroid, P., Nath, S., Santhiar, A., Thummalapenta, S., 2019a. Root causing flaky tests in a large-scale industrial setting, in: Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 101–111.

Lam, W., Muşlu, K., Sajnani, H., Thummalapenta, S., 2020. A study on the lifecycle of flaky tests, in: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering, pp. 1471–1482.

Lam, W., Oei, R., Shi, A., Marinov, D., Xie, T., 2019b. idflakies: A framework for detecting and partially classifying flaky tests, in: Proceedings of the 12th IEEE International Conference on Software Testing, Verification and Validation (ICST 2019), IEEE. pp. 312–322. doi:10.1109/ICST.2019.00038.

Luo, Q., Hariri, F., Eloussi, L., Marinov, D., 2014. An empirical analysis of flaky tests, in: Proceedings of the 22nd ACM SIGSOFT Symposium on the Foundations of Software Engineering (FSE '14), ACM. pp. 643–653.

Matsakis, N.D., Klock, F.S., 2014. The rust language, in: Proceedings of the 2014 ACM SIGAda annual conference on High integrity language technology, pp. 103–104.

Memon, A., Gao, Z., Nguyen, B., Dhanda, S., Nickell, E., Siemborski, R., Micco, J., 2017. Taming google-scale continuous testing, in: Proceedings of the 2017 IEEE/ACM 39th International Conference on Software Engineering: Software Engineering in Practice Track (ICSE-SEIP), IEEE. pp. 233–242.

Micco, J., 2017. The state of continuous integration testing @Google. URL: https://research.google/pubs/the-state-of-continuous-integration-testing-google/.

Murillo, J.M., Garcia-Alonso, J., Moguel, E., Barzen, J., Leymann, F., Ali, S., Yue, T., Arcaini, P., Pérez-Castillo, R., García-Rodríguez de Guzmán, I., Piattini, M., Ruiz-Cortés, A., Brogi, A., Zhao, J., Miranskyy, A., Wimmer, M., 2025. Quantum software engineering: Roadmap and challenges ahead. ACM Transactions on Software Engineering and Methodology 34. doi:10.1145/3712002.

Parry, O., Kapfhammer, G.M., Hilton, M., McMinn, P., 2021. A survey of flaky tests. ACM Transactions on Software Engineering and Methodology (TOSEM) 31, 1–74.

Pinto, G., Miranda, B., Dissanayake, S., d'Amorim, M., Treude, C., Bertolino, A., 2020. What is the vocabulary of flaky tests?, in: Proceedings of the 17th International Conference on Mining Software Repositories (MSR '20), ACM/IEEE. pp. 492–502. doi:10.1145/3379597.3387482.

Qiskit Development Community, 2025a. Configure qiskit locally. URL: https://quantum.cloud.ibm.com/docs/en/guides/configure-qiskit-local. [Online; accessed 2025-11-07].

Qiskit Development Community, 2025b. Qiskit basicaer provider (archived 0.45) documentation. URL: https://github.com/Qiskit/documentation/blob/archived-docs/docs/api/qiskit/0.45/providers_basicaer.mdx. [Online; accessed 2025-11-07].

Qiskit Development Community, 2025c. Qiskit circuit api documentation. URL: https://quantum.cloud.ibm.com/docs/en/api/qiskit/circuit. [Online; accessed 2025-11-07].

Qiskit Development Community, 2025d. Qiskit compiler api documentation. URL: https://quantum.cloud.ibm.com/docs/en/api/qiskit/compiler. [Online; accessed 2025-11-07].

Qiskit Development Community, 2025e. Qiskit operators and quantum information overview. URL: https://quantum.cloud.ibm.com/docs/en/guides/operators-overview. [Online; accessed 2025-11-07].

Qiskit Development Community, 2025f. Qiskit primitives api documentation. URL: https://quantum.cloud.ibm.com/docs/en/api/qiskit/0.46/primitives. [Online; accessed 2025-11-07].

Qiskit Development Community, 2025g. Qiskit providers and backend interface documentation. URL: https://quantum.cloud.ibm.com/docs/. [Online; accessed 2025-11-07].

Qiskit Development Community, 2025h. Qiskit pulse api and guide. URL: https://quantum.cloud.ibm.com/docs/en/api/qiskit/pulse. [Online; accessed 2025-11-07].

Qiskit Development Community, 2025i. Qiskit qpy api documentation. URL: https://quantum.cloud.ibm.com/docs/en/api/qiskit/qpy. [Online; accessed 2025-11-07].

Qiskit Development Community, 2025j. Qiskit readme documentation. URL: https://github.com/Qiskit/qiskit/blob/25c8a9312b9f92d54c05c27aa32ecd30cae15301/README.md?plain=1#L15-L16.

Qiskit Development Community, 2025k. Qiskit repository and test_examples test suite. URL: https://github.com/Qiskit/qiskit. [Online; accessed 2025-11-07].

Qiskit Development Community, 2025l. Qiskit scheduler (archived 0.45) documentation. URL: https://github.com/Qiskit/documentation/blob/archived-docs/docs/api/qiskit/0.45/scheduler.mdx. [Online; accessed 2025-11-07].

Qiskit Development Community, 2025m. Qiskit synthesis api documentation. URL: https://quantum.cloud.ibm.com/docs/en/api/qiskit/synthesis. [Online; accessed 2025-11-07].

Qiskit Development Community, 2025n. Qiskit opflow (archived 0.45) documentation. URL: https://github.com/Qiskit/documentation/blob/archived-docs/docs/api/qiskit/0.45/opflow.mdx. [Online; accessed 2025-11-07].

Qiskit Development Community, 2025o. Qiskit `quantum_info` api documentation. URL: `https://quantum.cloud.ibm.com/docs/en/api/qiskit/quantum_info`. [Online; accessed 2025-11-07].

Qiskit Development Community, 2025p. Qiskit transpiler api documentation. URL: `https://quantum.cloud.ibm.com/docs/en/api/qiskit/transpiler`. [Online; accessed 2025-11-07].

Qiskit Development Community, 2025q. Qiskit visualization api documentation. URL: `https://quantum.cloud.ibm.com/docs/en/api/qiskit/visualization`. [Online; accessed 2025-11-07].

Qiskit Development Community, 2026a. Empirical snapshot of flaky tests in qiskit v2.2.3; feedback welcome · issue #15605 · qiskit/qiskit. URL: `https://github.com/Qiskit/qiskit/issues/15605`. [Online; accessed 2026-05-28].

Qiskit Development Community, 2026b. Seed randomness in 'ucr' tests by jakelishman · pull request #15651 · qiskit/qiskit. URL: `https://github.com/Qiskit/qiskit/pull/15651`. [Online; accessed 2026-05-28].

Sivaloganathan, J., Jamshidi, A., Miranskyy, A., Zhang, L., 2024. Automating quantum software maintenance: Flakiness detection and root cause analysis. arXiv preprint arXiv:2410.23578 .

Sivaloganathan, J., Jamshidi, A., Miranskyy, A., Zhang, L., 2026. Automating detection and root-cause analysis of flaky tests in quantum software. arXiv preprint arXiv:2603.09029 .

Tox Development Community, . tox-dev/tox: Command line driven CI frontend and development task automation tool. URL: `https://github.com/tox-dev/tox`.

Verdecchia, R., Cruciani, E., Miranda, B., Bertolino, A., 2021. Know you neighbor: Fast static prediction of test flakiness. IEEE Access 9, 76119–76134.

Wieringa, R.J., Daneva, M., 2015. Six strategies for generalizing software engineering theories. Science of computer programming 101, 136–152. doi:`10.1016/J.SCICO.2014.11.013`.

Wilson, E.B., 1927. Probable inference, the law of succession, and statistical inference. Journal of the American Statistical Association 22, 209–212.

Wohlin, C., Runeson, P., Höst, M., Ohlsson, M., Regnell, B., Wesslén, A., 2012. Experimentation in Software Engineering. Computer Science, Springer Berlin Heidelberg.

Yin, R., 2009. Case Study Research: Design and Methods. Applied Social Research Methods, SAGE Publications.

Zhang, L., Miranskyy, A., 2024. Automated flakiness detection in quantum software bug reports, in: 2024 IEEE International Conference on Quantum Computing and Engineering (QCE), IEEE. pp. 179–181.

Zhang, L., Radnejad, M., Miranskyy, A., 2023. Identifying flakiness in quantum programs, in: Proceedings of the 17th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM 2023), ACM/IEEE. doi:10.1109/ESEM56168.2023.10304850.

Ziftci, C., Cavalcanti, D., 2020. De-flake your tests: Automatically locating root causes of flaky tests in code at Google, in: 2020 IEEE International Conference on Software Maintenance and Evolution (ICSME), IEEE. pp. 736–745.

Appendix A. Module taxonomy

Below are descriptions of the main subcomponents of Qiskit TERRA.

`transpiler` Tests passes, preset pass managers, layout, routing, basis translation, scheduling, and `transpile()` behavior. This subcomponent is present and actively maintained across all studied releases (0.25.0–1.2.4) (Qiskit Development Community, 2025p).

`quantum_info` Tests the state/operator/channel toolbox (e.g., `Statevector`, `DensityMatrix`, `SparsePauliOp`), including conversions, algebraic operations, metrics, and channel properties. Available throughout 0.25.0–1.2.4 (Qiskit Development Community, 2025e,o).

`compiler` Tests high-level compile wrappers (e.g., `transpile`, `assemble`) that orchestrate transpilation, scheduling, and backend configuration. Present as a thin wrapper interface in all considered versions (Qiskit Development Community, 2025d).

`synthesis` Tests unitary and operator synthesis routines that decompose targets into supported gate sets (exact and approximate). Supported across 0.25.0–1.2.4 (Qiskit Development Community, 2025m).

`circuit` Tests the core `QuantumCircuit` abstraction, gate and instruction definitions, parameterization, control-flow, and circuit transformation APIs. Central and stable over 0.25.0–1.2.4 (Qiskit Development Community, 2025c).

`qpy` Tests QPY-based serialization and deserialization of circuits, checking round-trip correctness and compatibility. Used across all studied releases (Qiskit Development Community, 2025i).

`pulse` Tests pulse-level programming (channels, waveforms, schedule blocks) and alignment with backend timing/constraints. Available throughout 0.25.0–1.2.4, though some features are later streamlined (Qiskit Development Community, 2025h).

`scheduler` Tests legacy helpers converting circuits to scheduled pulse programs, including timing and resource constraints. Relevant only to earlier releases in our range (removed around 0.45) (Qiskit Development Community, 2025l).

`visualization` Tests circuit drawers, state plots, device/heatmap plots, and related options. Present from 0.25.0 to 1.2.4 (Qiskit Development Community, 2025q).

`providers` Tests provider and backend interfaces (including fake backends), job submission, configuration, and result formats. Present across all studied versions (Qiskit Development Community, 2025g).

`test_user_config` Tests loading and applying user configuration (e.g., defaults, visualization settings) from local config files. Present in our full range (Qiskit Development Community, 2025a).

`basicaer` Tests legacy local simulators for correctness and compatibility. Only applicable to the subset 0.25.0–0.46.x within our window (Qiskit Development Community, 2025b).

`opflow` Tests operator-flow abstractions for operator expressions, expectations, and gradients (now legacy). Appears in 0.25.0–0.46.x within our range (Qiskit Development Community, 2025n).

`primitives` Tests the `Sampler` and `Estimator` primitives, including interfaces, result formats, and backend/runtime integration. Becomes relevant from roughly 0.39.0 and remains central through 1.2.4 (Qiskit Development Community, 2025f).

`test_examples` Tests example scripts and tutorials to ensure that published examples run end-to-end against the public API. Present throughout 0.25.0–1.2.4 (Qiskit Development Community, 2025k).